

Python
for DA

Revision. Pandas



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

- 

Камера должна быть включена на протяжении всего занятия
- 

В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
- 

Вести себя уважительно и этично по отношению к остальным участникам занятия
- 

Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
- 

Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя

Повторение

-  Создание массивов в NumPy
-  Атрибуты массивов в NumPy
-  Инициализация массивов в NumPy
-  Создание последовательностей с помощью `np.arange()` и `np.linspace()`
-  Арифметические операции в NumPy
-  Операции над матрицами
-  Изменение формы и размера массивов в NumPy
-  Случайные числа в NumPy

План занятия

- Серии в Pandas
- DataFrame в Pandas
- Очистка данных в Pandas
- Слияние и объединение в Pandas
- Группировка данных и сводные таблицы в Pandas



ОСНОВНОЙ БЛОК





Серии в Pandas



Серия Pandas

Это одномерный маркированный массив, способный хранить данные любого типа. Построен поверх массивов NumPy и предоставляет дополнительную функциональность, такую как индексация по меткам и выравнивание данных.

Создание серии Pandas



Непосредственно из списка или массива



С помощью функций NumPy



Из словаря

```
import pandas as pd

data = [10, 20, 30, 40, 50]
series = pd.Series(data)
print(series)
```

Пример: Создание серии из списка

```
0    10
1    20
2    30
3    40
4    50
dtype: int64
```

Вывод:

```
import numpy as np

# Использование numpy.linspace()
ser1 = pd.Series(np.linspace(3, 33, 3))
print(ser1)

# Использование numpy.random.normal()
ser2 = pd.Series(np.random.normal(0.0, 1.0,
5))
print(ser2)
```

Пример: Создание серии с помощью функций NumPy

```
0      3.0
1     18.0
2     33.0
dtype: float64
0      0.178420
1     -0.548335
2      0.604626
3     -0.058498
4      0.354780
dtype: float64
```

Вывод

```
series1 = pd.Series([1, 2, 3, 4, 5])
series2 = pd.Series([10, 20, 30, 40, 50])

# Сложение
result= series1 + series2
print(result)
```

Пример: Арифметические операции

```
0    11
1    22
2    33
3    44
4    55
dtype: int64
```

Вывод

```
series = pd.Series([1, 2, 3, 4, 5])
```

```
# Сумма
print(series.sum())
```

```
# Среднее
print(series.mean())
```

```
15
3.0
```

Пример: Агрегация

Выходные данные

```
series = pd.Series([10, 20, 30, 40, 50])

# Доступ к первому элементу
print(series[0])

# Нарезка
print(series[1:4])
```

Пример: Позиционное индексирование

```
10
1    20
2    30
3    40
dtype: int64
```

Выходные данные

```
series = pd.Series([10, 20, 30, 40, 50],
index=['a', 'b', 'c', 'd', 'e'])
```

```
# Доступ к элементу с меткой 'b'
print(series['b'])
```

```
# Нарезка с метками
print(series['b':'d'])
```

```
20
b    20
c    30
d    40
dtype: int64
```

Пример: Индексация на основе меток

Выходные данные



ВОПРОСЫ





DataFrame в Pandas



Pandas DataFrame

Это двумерная, изменяемая по размеру и потенциально неоднородная табличная структура данных с маркированными осями. Каждый столбец в DataFrame может содержать различные типы данных.

```
import pandas as pd

data = {
    "Name": ["Том", "Ник", "Криш", "Джек"],
    "Возраст": [20, 21, 19, 18]
}
df = pd.DataFrame(data)
print(df)
```

Пример: Создание фрейма данных из словаря

	Name	Возраст
0	Том	20
1	Ник	21
2	Криш	19
3	Джек	18

Вывод

```
data = [['Tom', 20], ['Nick', 21], ['Krish',
19], ['Jack', 18]]
df = pd.DataFrame(data, columns=['Name',
'Age'])
print(df)
```

Пример: Создание фрейма данных из списка списков

	Name	Age
0	Tom	20
1	Nick	21
2	Krish	19
3	Jack	18

Вывод

```
import numpy as np

data = np.array(['', 'Col1', 'Col2'],
                ['Row1', 1, 2], ['Row2', 3, 4])
df = pd.DataFrame(data=data[1:, 1:],
                  index=data[1:, 0], columns=data[0, 1:])
print(df)
```

Пример: Создание фрейма данных из массива NumPy

	Col1	Col2
Row1	1	2
Row2	3	4

Вывод

Пример: Создание фрейма данных из файла CSV

```
df = pd.read_csv('data.csv')  
print(df)
```

```
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = pd.DataFrame({'A': [10, 20, 30], 'B': [40, 50, 60]})

# Сложение
result= df1 + df2
print(result)
```

Пример: Арифметические операции

	A	B
0	11	44
1	22	55
2	33	66

Вывод

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
```

```
# Сумма
print(df.sum())
```

```
# Среднее
print(df.mean())
```

Пример: Агрегация

```
A      6
B     15
dtype: int64
A      2.0
B      5.0
dtype: float64
```

Вывод


```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
```

```
# Доступ к первому ряду
print(df.iloc[0])
```

```
# Нарезка строк
print(df.iloc[1:3])
```

Пример: Позиционное индексирование

```
A      1
B      4
Name: 0, dtype: int64
   A  B
1  2  5
2  3  6
```

Вывод

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]}, index=['row1', 'row2', 'row3'])

# Доступ к ряду по метке
print(df.loc['row2'])

# Нарезка строк по меткам
print(df.loc['row1':'row2'])
```

Пример: Индексирование на основе меток

```
A      2
B      5
Name: row2, dtype: int64

   A  B
row1 1  4
row2 2  5
```

Вывод

Атрибуты фреймов данных



`df.shape`: Возвращает кортеж, представляющий размерность фрейма данных.



`df.columns`: Возвращает метки столбцов DataFrame.



`df.index`: Возвращает метки строк фрейма DataFrame.

Методы фреймов данных



`df.head(n)`: Возвращает первые `n` строк фрейма `DataFrame`.



`df.tail(n)`: Возвращает последние `n` строк фрейма `DataFrame`.



`df.describe()`: Генерирует описательную статистику фрейма `DataFrame`.



`df.info()`: Предоставляет краткую информацию о `DataFrame`.



`df.drop(columns)`: Удаляет указанные столбцы из `DataFrame`.



`df.rename(columns)`: Переименовывает столбцы в `DataFrame`.

```
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
```

```
# Форма фрейма данных
print(df.shape)
```

```
# Имена столбцов
print(df.columns)
```

```
# Первые две строки
print(df.head(2))
```

```
# Описательная статистика
print(df.describe())
```

Пример: Использование атрибутов и методов

```
(3, 2)
Index(['A', 'B'], dtype='object')
   A  B
0  1  4
1  2  5
```

	A	B
count	3.0	3.0
mean	2.0	5.0
std	1.0	1.0
min	1.0	4.0
25%	1.5	4.5
50%	2.0	5.0
75%	2.5	5.5
max	3.0	6.0

Вывод



ВОПРОСЫ





Очистка данных в Pandas



Очистка данных

Это важнейший этап процесса анализа данных, включающий выявление и устранение ошибок, несоответствий и недостающих значений в наборе данных. Гарантирует, что данные точны, надежны и готовы к анализу или моделированию.

Очистка данных в Pandas

Опускание пропущенных значений

```
df_cleaned = df.dropna()
```

Заполнение пропущенных значений

```
df_filled = df.fillna(df.mean())
```

Удаление дубликатов

```
df_no_duplicates = df.drop_duplicates()
```

Исправление типов данных

```
df['column'] = df['column'].astype('int')
```

Переименование столбцов

```
df.rename(columns={'old_name': 'new_name'}, inplace=True)
```

Исследовательский анализ данных (EDA)

```
print(df.info())  
print(df.describe())  
print(df.head())
```

```
import pandas as pd

data = {'A': [1, 2, 3, None, 5], 'B': [None,
2, 3, 4, 5], 'C': [1, 2, None, None, 5]}
df = pd.DataFrame(data)
df_cleaned = df.dropna()
print(df_cleaned)
```

Пример: Удаление строк с пропущенными значениями

	A	B	C
1	2.0	2.0	2.0
4	5.0	5.0	5.0

Вывод

```
df_filled = df.fillna(df.mean())
print(df_filled)
```

Пример: Заполнение пропущенных значений средним значением

	A	B	C
0	1.00	3.5	1.000000
1	2.00	2.0	2.000000
2	3.00	3.0	2.666667
3	2.75	4.0	2.666667
4	5.00	5.0	5.000000

Вывод

```
df_no_duplicates = df.drop_duplicates()
print(df_no_duplicates)
```

Пример: Удаление дублирующихся строк

```
df.rename(columns={'A': 'Column_A', 'B':
'Column_B'}, inplace=True)
print(df)
```

Пример: Переименование столбцов



ВОПРОСЫ





Слияние и объединение в Pandas



Слияние и объединение

Это операции при работе с данными, позволяющие объединить несколько фреймов данных на основе общих столбцов или индексов.



merge()

Это функция, которая является основным инструментом для объединения DataFrames. Поддерживает различные типы объединений, включая внутренние, внешние, левые и правые объединения.

```
import pandas as pd

df1 = pd.DataFrame({'a': ['a1', 'a2', 'a3'],
                    'b': ['b1', 'b2', 'b3']})
df2 = pd.DataFrame({'a': ['a1', 'a2', 'a4'],
                    'c': ['c1', 'c2', 'c4']})

merged_df = pd.merge(df1, df2, on='a')
print(merged_df)
```

Пример: Слияние фреймов данных

	a	b	c
0	a1	b1	c1
1	a2	b2	c2

Вывод

Типы объединений в merge()

Внутреннее соединение

```
pd.merge(df1, df2, how='inner', on='a')
```

Левое объединение

```
pd.merge(df1, df2, how='left', on='a')
```

Правое объединение

```
pd.merge(df1, df2, how='right', on='a')
```

Внешнее объединение

```
pd.merge(df1, df2, how='outer', on='a')
```

```
df2 = df2.rename(columns={'a': 'a_'})
merged_df = pd.merge(df1, df2, left_on='a',
right_on='a_')
print(merged_df)
```

Пример: Указание столбцов для слияния

	a	b	a_	c
0	a1	b1	a1	c1
1	a2	b2	a2	c2

Вывод



join()

Это метод, метод экземпляра, который объединяет фреймы DataFrames на основе их индексов. Это более простой и эффективный способ объединения DataFrames по сравнению с `merge()`, когда вы работаете с индексами.

```
df1 = pd.DataFrame({'a': ['a1', 'a2', 'a3']},
index=['i1', 'i2', 'i3'])
df2 = pd.DataFrame({'b': ['b1', 'b2', 'b3']},
index=['i1', 'i2', 'i4'])

joined_df = df1.join(df2, how='left')
print(joined_df)
```

Пример: Базовое использование join()

	a	b
i1	a1	b1
i2	a2	b2
i3	a3	NaN

Вывод

Типы объединений в join()

Left Join

```
df1.join(df2, how='left')
```

Right Join

```
df1.join(df2, how='right')
```

Внутреннее соединение

```
df1.join(df2, how='inner')
```

Внешнее соединение

```
df1.join(df2, how='outer')
```

```
df1 = pd.DataFrame({'a': ['a1', 'a2', 'a3']})
df2 = pd.DataFrame({'b': ['b1', 'b2', 'b3']})

concatenated_df = pd.concat([df1, df2],
axis=1)
print(concatenated_df)
```

Пример: Базовое использование функции
concat()

	a	b
0	a1	b1
1	a2	b2
2	a3	b3

Вывод


```
concatenated_df = pd.concat([df1, df2],
keys=['df1', 'df2'])
print(concatenated_df)
```

Пример: Конкатенация с ключами

		a	b
df1	0	a1	NaN
	1	a2	NaN
	2	a3	NaN
df2	0	NaN	b1
	1	NaN	b2
	2	NaN	b3

Вывод



ВОПРОСЫ





Группировка данных и сводные таблицы в Pandas



Группировка данных и создание сводных таблиц

Это операции при работе с данными, позволяющие объединить несколько фреймов данных на основе общих столбцов или индексов.

```
import pandas as pd

# Образец данных
data = {
    'Category': ['A', 'B', 'A', 'B', 'A', 'B'],
    'Value': [10, 20, 30, 40, 50, 60]
}
df = pd.DataFrame(data)

# Группируем по "Category" и вычисляем сумму "Value"
grouped_df = df.groupby('Category').sum()
print(grouped_df)
```

Пример: Базовое использование функции `groupby()`

Category	Value
A	90
B	120

Вывод

```
# Группировка по 'Category' и вычисление
нескольких агрегированных значений
grouped_df =
df.groupby('Category').agg({'Value': ['sum',
'mean', 'max']})
print(grouped_df)
```

Пример: Расширенная группировка

	Value		
	sum	mean	max
Category			
A	90	30.0	50
B	120	40.0	60

Вывод

```
# Образец данных
data = {
    'A': ['foo', 'foo', 'foo', 'bar', 'bar', 'bar'],
    'B': ['one', 'one', 'two', 'two', 'one', 'one'],
    'C': ['маленький', 'большой', 'маленький',
          'маленький', 'большой', 'маленький'],
    'D': [1, 2, 2, 3, 3, 4]
}
df = pd.DataFrame(data)

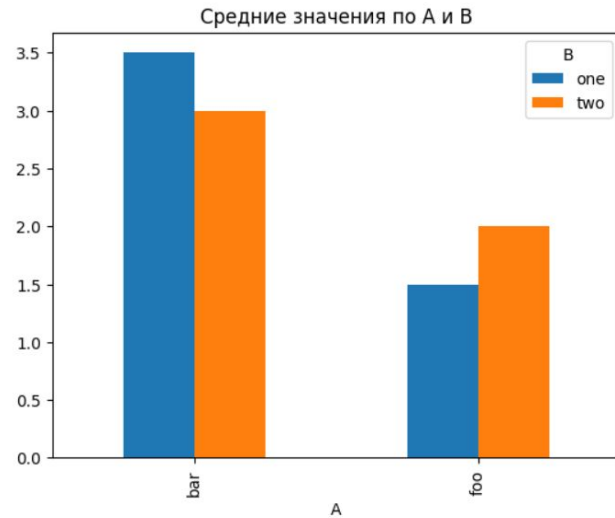
# Создание сводной таблицы
pivot_table = pd.pivot_table(df, values='D', index=['A',
'B'], columns=['C'], aggfunc='sum', fill_value=0)
print(pivot_table)
```

Пример: Создание сводных таблиц с помощью функции `pivot_table()`

C		большой	маленький
A	B		
bar	one	3	4
	two	0	3
foo	one	2	1
	two	0	2

Вывод

```
# Построение графиков по таблице pivot
pivot_table = pd.pivot_table(df, values='D',
                              index='A', columns='B', aggfunc='mean')
pivot_table.plot(kind='bar', title='Средние значения по A и B')
```



Пример: Построение графиков в таблицах Pivot

Вывод


```

pivot_table = pd.pivot_table(df,
values=['D'], index=['A', 'C'], aggfunc={'D':
['mean', 'sum', 'max']})
print(pivot_table)

```

Пример с несколькими агрегациями

A	C	D		
		max	mean	sum
bar	большой	3	3.0	3
	маленький	4	3.5	7
foo	большой	2	2.0	2
	маленький	2	1.5	3

Вывод



ВОПРОСЫ





ПРАКТИЧЕСКАЯ РАБОТА



Задание



1. Создайте DataFrame из словаря, добавьте новый столбец и вычислите среднее значение столбца.
2. Фильтруйте строки на основе условия и сортируйте DataFrame по определенному столбцу.



ВОПРОСЫ



Домашнее задание

1. Подгоните полином к набору точек данных:

```
x = np.linspace(-10, 10, N)
y = 3 * x**3 + 2 * x**2 + x + np.random.normal(0, 100, N)
```

и постройте график результатов. Используйте функцию `np.polyfit`.

2. Создайте возрастные группы и рассчитайте коэффициент выживаемости для каждой комбинации возрастной группы и класса в наборе данных Titanic.

Заключение

